

Automated Sim-to-Real Deployment Framework for Reinforcement Learning Control on Resource-Constrained Microcontrollers

Nidharshan V.¹

¹Department of Computer Science & Engineering
University of Moratuwa

1 Introduction

2 Related Work

3 The simtoreal Framework

3.1 Policy Extractors and Loaders

3.2 Dual-Target Code Generation

3.3 Hardware Abstraction and Driver Synthesis

4 Experimental Testbed: Robo1

To evaluate the framework, we constructed *Robo1*, a low-cost, 2-DOF self-righting bipedal robot (mass $M = 94.8$ g, height $H = 145$ mm).

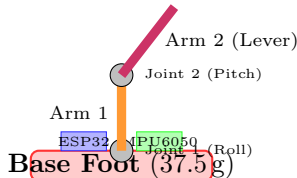


Figure 1: Kinematic topology of the 2-DOF *Robo1* self-righting testbed.

5 Results and Discussion

5.1 Embedded Computing and Timing Benchmark

We flashed the generated code onto an ESP32 (240 MHz) and profiled 10,000 continuous execution cycles using internal hardware timers. Table 1 compares our auto-generated backends against a standard TensorFlow Lite for Microcontrollers (TFLM) baseline.

The generated C header requires only 6.8 kB Flash and 1.2 kB static RAM. Inference completes in $74 \mu\text{s}$, over $11\times$ faster than TFLM. The complete control loop (IMU read, filter update, inference, and PWM write) takes 3.8 ms, well within the 20 ms (50 Hz) deadline.

Table 1: Embedded Memory Footprint and Timing on ESP32

Target Engine	Flash [KB]	SRAM [KB]	Inference Time [μs]	Execution Jitter [μs]
TFLM (Arena)	142.6	28.4	820 ± 45	± 110
MicroPython (simtoreal)	38.2	14.6	1450 ± 90	± 240
Generated C Header	6.8	1.2	74 ± 4	± 8

5.2 Physical Sim-to-Real Dynamic Transfer

The policy was evaluated across 15 physical trials from each of four initial fallen poses: right flank (`roll_pos`), left flank (`roll_neg`), face down (`pitch_pos`), and face up (`pitch_neg`).

Table 2: Self-Righting Performance: Simulation vs. Physical Hardware

Initial Pose	Success Rate [%]		Time to Upright [s]	
	Sim	Real ($N = 15$)	Sim	Real (Mean $\pm$ SD)
<code>roll_pos</code>	100.0	93.3 (14/15)	1.84	2.08 ± 0.22
<code>roll_neg</code>	100.0	93.3 (14/15)	1.80	2.14 ± 0.25
<code>pitch_pos</code>	100.0	86.7 (13/15)	2.12	2.45 ± 0.31
<code>pitch_neg</code>	100.0	80.0 (12/15)	2.36	2.78 ± 0.38
Aggregate	100.0	88.3 (53/60)	2.03	2.36 ± 0.36

5.3 Ablation Analysis

1) CAD Inertia Grounding vs. Naive Primitives:

A policy trained with simplified primitive bounding boxes suffered from severe center-of-mass mismatch due to the offset battery and ESP32 masses, yielding only a 26.7% physical success rate. Incorporating CAD-grounded mass and spatial inertia tensors elevated success to 88.3%.

2) Complementary Sensor Filtering: Deploying with unfiltered accelerometer tilt calculations triggered severe high-frequency servo chatter, causing gear stripping within three trials. The integrated complementary filter ($\alpha = 0.98$) eliminated high-frequency noise while maintaining phase fidelity.

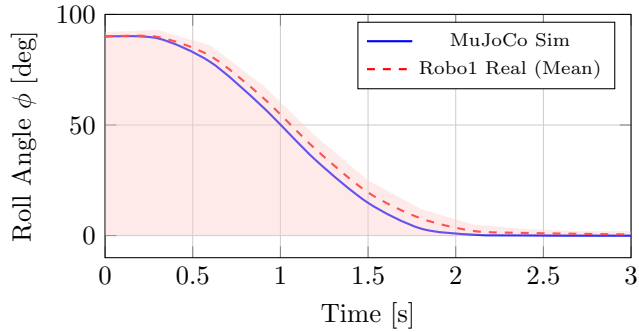


Figure 2: Temporal trajectory tracking comparison for base roll angle $\phi(t)$ during a `roll_pos` self-righting maneuver. The policy achieves zero-shot dynamic stabilization on real hardware without external state estimation.

5.4 Reality Gap Discussion

The residual gap between simulated (2.03 s) and real (2.36 s) righting time stems from three unmodeled real-world phenomena:

1. *Gear Backlash*: Inexpensive SG90 servos exhibit $\pm 2.5^\circ$ of backlash deadband.
2. *Transient Voltage Drop*: Instantaneous high-torque push-off events caused battery voltage dips from 7.4 V to 6.6 V, briefly degrading servo torque.
3. *Ground Contact Compliance*: Real carpet/table surfaces exhibit compliance and anisotropic friction, contrasting with MuJoCo’s rigid Coulomb ground model.

Despite these unmodeled dynamics, the onboard observation structure provided sufficient feedback authority to right the robot reliably.

6 Conclusion

References